Project 1 - MLA210

Spring 2024

The submission deadline is 22.03.2024

You are encouraged to collaborate two and two (larger groups are not allowed).

Write your NAME(s) here: Milad Tootoonchi

Working process & ethics:
The important process of learning through problemsolving includes trial, error and exploration. It is therefore OK to discuss with other fellow students to share ideas and experiences - **BUT** copying the work of others **is strictly forbidden.**

Note:
You can start with parts A and D already now.

The present project will need access to **the following Julia packages**:

In [1]:
```
using VMLS
using LinearAlgebra
using Plots
using Random
```

Part A: Warm up (7 short exercises).

Ex. A1: The **arithmetic mean)** (average) value μ of an n-dimensional vector x can be calculated by the formula $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$.

From linear algebra theory we know that with 1_n denoting the n-dimensional vector with ones (1) i all positions, the summation $\sum_{i=1}^{n} x_i$ can be calculated by the inner product $1_n^T x$, thus $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{1}{n}(1_n^T x)$.

From Julia programming we also know that the dimension (n) of a vector x can be found by `n = size(x,1)`, and that an n-vector of ones is given by `ones(n)`.

Use this information and define your own Julia function `mymean(x)` to calculate the mean value of vectors. Fill in the required line(s) of code in the following "skeleton" to provide your solution.

In [2]:
```
function mymean(x)
    n = size(x,1);
    one = ones(n)
```

```
    μ = 1/n * (one' * x)
    return μ
end
```

```
mymean (generic function with 1 method)
```

It's a good idea to check that your `mymean`-function is working correctly by comparing its calculations to the `avg(x)` -function in the VMLS-package for some real /random vectors:

Ex. A2: Explain the following lines of code by making informative comments (comments in Norwegian is fine):

In [ ]:
```
# <... fill in introduction comment ....>:
nT = 10000; # the variable nT is set to 10000
mymeanTest = zeros(nT, 2);
# mymeantest variable is set to a nT x 2 matrix with zeros,
#or 10000 x 2 (since nT is 10000)

for k = 1:nT
  # starting a for loop with k from 1 to 10000 (nT = 10000)
  n = rand(1:1000); # n is set to a random number between 1 and 1000
  x = n*rand(n);
  # x is set to n times a vector with n elements
  #of random numbers between 0 and 1.
  mymeanTest[k,1] = mymean(x);
  # testing our custom mean function,
  #mean of x is set to row k, column 1 of the matrix mymeanTest
  mymeanTest[k,2] = avg(x);
  # putting the average of x to row k, column 2 of the matrix mymeanTest
end

dT = mymeanTest[:,1]-mymeanTest[:,2];
# dT is set to a vector with the difference of the our -
# calculatet means and averages of x
norm(dT)  # returns the norm of dT, should be ~ 0
```

```
4.323916918287546e-12
```

Ex. A3: The **standard deviation** of the values in an n-dimensional vector x can be calculated by the formula $\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$, where μ is the (arithmetic) mean defined above.

Use this to define your own Julia function `mystd(x)` that calculates the standard deviation of vectors. Fill in the required line(s) of code in the "skeleton" below to provide your solution.

Hint: The statement `(x.-μ).^2` in Julia computes the squared differences $(x_i - \mu)^2$ as a vector, and use this information and your `mymean`-function to calculate $\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$. For calculating the square root of any number `t` in Julia you can use the square root function `sqrt(t)`.

In [ ]:
```
function mystd(x)
    μ = mymean(x)
```

```
    sd = (x.-μ).^2  # squared differences

    n = size(x,1);
    one = ones(n)

    sum_sd = one' * sd  # sum of squared differences

    σ = sqrt((1/n) * sum_sd)

    return σ
end
```

```
mystd (generic function with 1 method)
```

Ex. A4: Now, it's required to check that your `mystd` -function is working correctly by comparing its calculations to the `stdev(x)` -function in the VMLS-package for some real /random vectors. Copy and modify the code from exercise 2. above to do the comparisons.

In [5]:
```
# This script tests the accuracy of the mystd function by comparing its output
# with the built-in stdev function for many randomly generated vectors.

nT = 10000;
mystdTest = zeros(nT, 2);

for k = 1:nT
  n = rand(1:1000);
  x = n*rand(n);
  mystdTest[k,1] = mystd(x);
  mystdTest[k,2] = stdev(x);
end

dT = mystdTest[:,1]-mystdTest[:,2];
norm(dT)
```

```
2.6983753963852593e-12
```

The **correlation coefficient between two n-dimensional vectors x and y** can be defined as $\rho = \frac{\sum_{i=1}^{n}(x_i-\mu_x)(y_i-\mu_y)}{\sqrt{\sum_{i=1}^{n}(x_i-\mu_x)^2 \sum_{i=1}^{n}(y_i-\mu_y)^2}}$, where μ_x and μ_y are the arithmetic means of x and y, respectively.

Ex. A5: Use this to define your own Julia function `mycorr(x,y)` that calculates the correlation coefficient between two n-dimensional vectors x and y without coding the summation formulas in the definitions. Fill in the required line(s) of code in the "skeleton" below to provide your solution.

In [6]:
```
function mycorr(x,y)
    # The numerator
    μ_x = mymean(x)
    μ_y = mymean(y)
    num = dot(x .- μ_x, y .- μ_y)

    # The demnominator
    den = sqrt(sum((x .- μ_x).^2) * sum((y .- μ_y).^2))
```

```
    ρ = num / den
    return ρ
end
```

```
mycorr (generic function with 1 method)
```

Ex A6: Recall that the **cosine of the angle** θ between two n-dimensional vectors x and y (see also in VMLS, page 57) is given by the definition $\cos(\theta) = \frac{x^T y}{\|x\|\|y\|}$. Define your own Julia function `mycos(x,y)` by filling in the required line(s) of code in the following "skeleton":

In [ ]:
```
function mycos(x, y)
    nx = sqrt(sum(x.^2))    # length of x
    ny = sqrt(sum(y.^2))    # length of y
    cosθ = (x' * y) / (nx * ny)
    return cosθ
end
```

```
mycos (generic function with 1 method)
```

Denote the **mean centered** versions of the vectors x and y as $x_c = x - \mu_x$ and $y_c = y - \mu_y$, respectively.

Ex. A7: Verify numerically that **the correlation coefficient** between x and y is identical to the **cosine of the angle** between the corresponding mean centered vectors x_c and y_c. Copy and modify the code from exercise 2. above to do the comparisons.

In [ ]:
```
# In this exercise, we verify numerically that the correlation coefficient between
# two vectors x and y is equal to the cosine of
#the angle between their mean-centered
# versions xc and yc. This is done by
#computing both quantities and comparing the results.

nT = 10000;
mycorrcosTest = zeros(nT, 2);

for k = 1:nT
    n = rand(2:1000);   # to avoid n = 1: zero-centered vectores
    x = n*rand(n);
    y = n*rand(n)

    # Mean-centered versions
    xc = x .- mymean(x)
    yc = y .- mymean(y)

    mycorrcosTest[k,1] = mycorr(xc, yc);
    mycorrcosTest[k,2] = mycos(xc, yc);
end

dT = mycorrcosTest[:,1] - mycorrcosTest[:,2];
norm(dT)
```

```
2.1072533179899133e-15
```

Part B: An alternative approach for estimating the linear least squares solution (3 short exercises).

Here we will work a little more with the **Sacramento house sales data** used in section 2.3 and chapter 13 of VMLS.

By using a constant vector of ones togeter with x_1, x_2 and x_3 described in the Julia-code below as the columns in the 774×4 feature data matrix X, we can compute the linear least squares solution of

$$Xb = y,$$

(where y is the selling price in the units of 1000 dollars for each of the recorded 774 house sales to be predicted by later applications of the resulting model) as follows:

In [ ]:
```
D = house_sales_data();
y = D["price"];     # price in thousand dollars.
n = length(y);

# The "original" features:
x1 = D["area"];     # area of house (in 1000ft^2).
x2 = D["beds"];     # number of bedrooms.
x3 = D["baths"];    # number of bathrooms.

X1 = [x1 x2 x3];    # The matrix of non-constant features

# The data matrix for modelling with a constant term:
X = [ones(n) X1];

# The least squares solution β of the system Xb = y is:
β = X\y;

# The fitted values (yhat) and the model residuals (r) based on the
#regression model are:
yhat = X*β;
r = (y-yhat);

# The RMS-value of this model is
rmsβ = rms(r)
```

```
74.8452980698705
```

Ex. B1: Compare this rms-value to the **rms**-value of the constant model by using the function `mystd` from **Part A**.

In [10]:
```
# Fill in to calculate the rms-value of the constant model by using mystd:
println("rms-value using mystd:                ", mystd(r))
println("difference between rms and mystd:     ", rmsβ - mystd(r))
```

```
rms-value using mystd:                74.8452980698705
difference between rms and mystd:     0.0
```

In the following we will focus on the matrix X_1 of non-constant column features (x_1, x_2 and x_3).

Note that your function `mymean` from **Part A** should also be capable of simultaneously calculating the mean value for all columns of any matrix, so that the result is a row vector of dimension equal to the number of matrix columns.

We denote the **centered version of** the vector y as y_s and the centered version of the matrix X_1 as X_s, where y_s and X_s are obtained by substracting the column means μ_y and μ_X from all entries in the corresponding columns of y and X_1 as follows:

In [ ]:
```
μy = mymean(y); μX = mymean(X1)
ys = y  .- μy;
Xs = X1 .- μX;
mymean(ys), mymean(Xs)
# Clearly, the column means of y0 and X0 are all effectively 0
```

```
(2.350115508457179e-15, [7.877706526151628e-16 -2.335197783061699e-16 2.983545079096
028e-17])
```

Ex. B2: Compute the linear least squares solution β_s of the mean centered system

$$X_s b = y_s.$$

Also calculate the fitted values $\hat{y}_s = X_s\beta_s$ and the rms_{β_s}-value of the model residuals $r_s = (y_s - \hat{y}_s)$. Compare rms_{β_s} to rms_β and the entries of β_s to the last three entries $\beta_{2:4}$ of the least squares solution β calculated above to conclude that they are identical (theoretically this is true for any linear least squares problem).

In [12]:
```
βs = Xs \ ys;       # Fill in to compute the least squares solution of Xs*b = ys.
yshat = Xs * βs;    # Fill in  to compute fitted values
rs = (ys-yshat);       # Fill in  to compute the residuals
rmsβs = rms(rs);    # Fill in  to compute the rms-value

# Compare rms-values and the regression coeff. entries:
rmsβ-rmsβs, norm(βs - β[2:4])
```

```
(0.0, 1.2784838787458764e-13)
```

Ex. B3: Verify numerically the identity
$\mu_y = \beta_1 + \mu_X\beta_s$,
where β_1 is the first entry of the least squares solution vector β of the system $Xb = y$, to conclude that both β_1 and $\beta_{2:4}$ (and hence β) can be calculated from the solution of the mean centered system (theoretically this is true for any linear least squares problem).

In [ ]:
```
β1 = mymean(ys) - mymean(Xs) * βs; # Fill in to compute β1

α = [β1; βs];    # All regression coeffs calculated from
#the mean centered problem:
[α β]            # Compare
```

```
4×2 Matrix{Float64}:
  -1.19426e-13   54.6692
 149.051        149.051
 -18.7369       -18.7369
  -0.596478      -0.596478
```

Part C: QR-factorization and projection matrices (3 short exercises).

As above we will assume that the matrix X has n rows and $p < n$ columns. In the notation of Part B, the normal equations for solving linear least squares problems are

$$X^T X b = X^T y,$$

and under the assumption that the X-columns are linearly independent $X^T X$ is invertible. The associated least squares solution is given by $\beta = (X^T X)^{-1} X^T y$.

We obtain the fitted values

$$\hat{y} = X\beta = X(X^T X)^{-1} X^T y = Py,$$

where the $n \times n$ matrix $P \overset{def}{=} X(X^T X)^{-1} X^T$ is called **the projection matrix** onto the column space of X, i.e. the fitted values $\hat{y}$ is the the result of projecting y onto the column space.

Ex. C1: Calculate the QR-factorization $QR = X$, and verify nummerically (using the **Sacramento house sales data**) that $H = QQ^T = X(X^T X)^{-1} X^T = P$.

In [ ]:
```
D = house_sales_data();
y = D["price"]; n = length(y);
x0 = ones(n); x1 = D["area"]; x2 = D["beds"]; x3 = D["baths"];
X = [x0 x1 x2 x3];

P   = X * inv(X' * X) * X'; # Projection matrix according
# to the definition above
Q,R = qr(X); Q = Matrix(Q);
H   = Q * Q';   # Projection matrix based on Q-matrix
#from the QR-factorization of X
norm(P - H) # Compare P and H by computing the norm of their difference.
```

```
1.2624868729785073e-14
```

The vector x_0 in the code above is the constant vector with ones (1) in all positions.

Ex. C2: Compute the projection matrix P_0 onto the columnspace of x_0, and explain the result of projecting y, i.e. describe the vector $P_0 y$. Also explain the QR-factorization of X_0.

In [15]:
```
# x0 is already defined as ones(n)
P₀ = x0 * inv(x0' * x0) * x0'   # Projection matrix onto span{x0}
P₀*y, mymean(y) # Compre these & comment.
```

```
([228.76944702842383, 228.76944702842383, 228.76944702842383, 228.76944702842383, 22
8.76944702842383, 228.76944702842383, 228.76944702842383, 228.76944702842383, 228.76
944702842383, 228.76944702842383  …  228.76944702842383, 228.76944702842383, 228.769
44702842383, 228.76944702842383, 228.76944702842383, 228.76944702842383, 228.7694470
2842383, 228.76944702842383, 228.76944702842383, 228.76944702842383], 228.7694470284
2377)
```

Projecting onto the column space of x_0 means projecting onto the space of constant vectors. When you apply this projection to y, the result is a vector where every entry equals the mean of y. In other words, the projection replaces the data with its average. The QR factorization of x_0 gives one normalized column and a single scalar, which together describe the same projection.

Ex. C3: Define $C = I_n - P_0$, where I_n is the $n \times n$ identity matrix. Describe the function of C by inspecting the result of the matrix multiplication CX_1.

This matrix has actually been calculated/used in a previous exercise - under what name?

In [16]:
```
C = Matrix(I, n, n) - P₀
CX1 = C*X1;
norm(CX1-Xs) # Compre & comment.
```

```
9.527949828541409e-14
```

The matrix C removes the mean component of a vector, leaving only the mean-centered part. Multiplying C with X_1 gives the same result as subtracting the column means from X_1. This is exactly the mean-centering matrix used earlier in the exercises.

Part D: k-means clustering - two alternative algorithms + one more (4 exercises).

In VMLS chapter 4.3 the **k-means clustering algorithm is described**, and we also wrote the following Julia-version of the algorithm for grouping data into k separate clusters:

In [17]:
```
function kmeans_(x, k; maxiters = 100, tol = 1e-5)
    # By writing kmeans_ instead of kmeans to display this implementation
    # INPUTS:  x - a set of vectors, k - the number of clusters to be calculated, m
    #          of iterations in the algorithm, tol - the (relative) tolerance limit
    # OUTPUTs: c - cluster labels assigned for each sample, z - the (k) cluster cen
    #          function values for each cluster iteration.

    N = length(x)      # Number of samples (vectors in x)
    d = length(x[1])  # dimension of the sample vectors.
    distances = zeros(N) # used to store the distance of each sample to the nearest

    z = [zeros(d) for j=1:k] # used to store representatives.
    Js = zeros(maxiters);    # For storing the objective function values of the pro
    # 'c' is an array of N integers between 1 and k. The initial cluster assignment
    c = [ rand(1:k) for i in 1:N ]

    Jprevious = Inf # used in stopping condition
```

```
    for iter = 1:maxiters

        # Cluster j representative is average of points in cluster j.
        for j = 1:k
            cj = [i for i=1:N if c[i] == j] # find the indices of the samples assoc
            z[j] = sum(x[cj]) / length(cj); # the updated center of cluster j (the
        end;

        # For each x[i], find distance to the nearest representative
        # and update its group index.
        for i = 1:N
            (distances[i], c[i]) =
                findmin([norm(x[i] - z[j]) for j = 1:k])
        end;

        # Compute clustering objective value of the current clustering.
        J = distances'*distances/N
        Js[iter] = J;
        # Show progress and terminate if J stopped decreasing or maximun number of
        println("Iteration ", iter, ": Jclust = ", J, ".")
        if (iter > 1 && abs(J - Jprevious) < tol * J) || iter == maxiters
            Js = Js[1:iter];
            return c, z, Js
        end
        Jprevious = J
    end
end
```

```
kmeans_ (generic function with 1 method)
```

Note: This algorithm is designed to operate on datasets where **the objects to be clustered are represented in a vector of vectors**.

An alternative implementation of the k-means algorithm

Consider the following alternative implementation of the k-means algorithm, where **the objects to be clustered are represented as the rows of a matrix**.

In [18]:
```
function newkmeans(X, k; tol = 5e-3)
# ------------- Another implementation of the k-means algorithm ------------------
# INPUT: ----------------------------------------------------------------------
# X        - data matrix (observed datapoints are rows in X).
# k        - the number of clusters (an integer >= 2, typically 2 to something smal
# OUTPUT: ---------------------------------------------------------------------
# C        - vector of final cluster labels (1,..,k) assigned to each datapoint.
# Z        - matrix of final cluster centers (given as rows)
# JS       - objective function values describing the clustering process (J small m
# cs       - the sizes of each cluster
# ---------------------------------------------------------------------------------
JS = [];                              # objective function values describing the mea
N      = size(X,1);                   # number of samples (rows) in tha data matrix
D2     = Array{Float64}(undef, N, k);# Matrix for storing distances between samples
C      = zeros(N);                    # N-vector of the cluster assignments
cs     = zeros(k);                    # k-vector to contain the cluster sizes
```

```
Jcurrent = 0; Jprevious = -1;     # Jcurrent and Jprevious represent objective func

ids = randperm(N)[1:k];           # Generate k random integers in [1, N] for random
Z   = copy(X[ids,:]);             # The initial randomy selected cluster centers ac

iter = 1;                         # For counting the number of iterations before co

while abs((Jcurrent-Jprevious)/Jprevious) > tol    # Repeat until convergence of th
    for i = 1:k # Here we calculate all the distances between X-rows and the cluste
        D2[:,i] = sum((X.-Z[[i],:]).^2, dims=2); # Squared euclidean distance for a
    end

    # Identify shortest distance and corresponding cluster number for each observat
    minD2, C = findmin(D2, dims=2); C = getindex.(C,2); # convert from CartesianInd
    # update old and new objective function values
    Jprevious = Jcurrent; Jcurrent = sum(minD2)/N; JS = [JS; Jcurrent];

    # Update the cluster centers based on the labelling of Cid:
    for i = 1:k
        rows_i = findall(vec(C.==i));                  # Find row-numbers of all c
        cs[i] = length(rows_i)                         # Number of samples in clus
        if cs[i]>0                                     # Update if i-th cluster is
        Z[i,:] = sum(X[rows_i,:],dims = 1)./cs[i]      # Update cluster centers as
        end
    end, println("Iteration ", iter, ": Jclust = ", Jcurrent, ".");
    iter +=1
end

return C, Z, JS, cs;
end
```

```
newkmeans (generic function with 1 method)
```

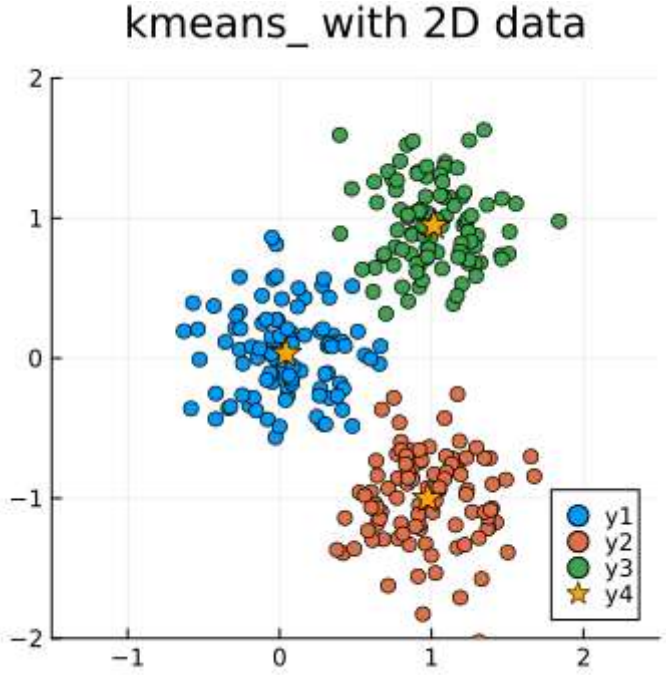
Here we run both algorithms on the same dataset:

In [19]:
```
# Generate a vector of random 2D-vectors for testing the two algorithms:
X = vcat( [ 0.3*randn(2) for i = 1:100 ],
[ [1,1] + 0.3*randn(2) for i = 1:100 ],
[ [1,-1] + 0.3*randn(2) for i = 1:100 ] );


k = 3;
c, z, J = kmeans_(X, k);   # k-means clustering by the above algorithm

# Scatter plot of the clustering solution
N = length(X)
scatter(title = "kmeans_ with 2D data",
    label = " ", legend = :bottomright, size = (350,350), xlims = (-1.5,2.5), ylims
for j = 1:k
    Cj = [X[i] for i=1:N if c[i] == j]
    scatter!([x[1] for x in Cj], [x[2] for x in Cj])
end
# Add cluster centers to the scatter plot:
scatter!([cs[1] for cs in z], [cs[2] for cs in z], marker = :star, markersize = 8,
```

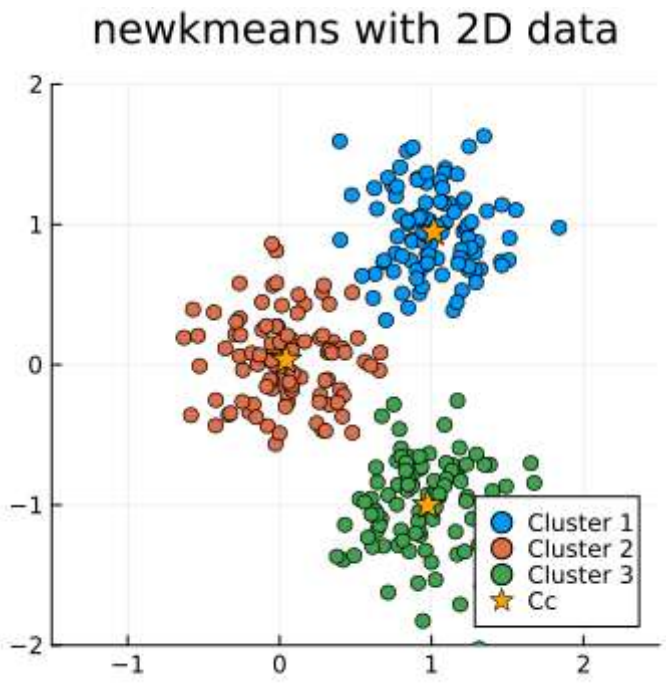
```
Iteration 1: Jclust = 0.905859296965941.
Iteration 2: Jclust = 0.2282427026764099.
Iteration 3: Jclust = 0.17515879126006698.
Iteration 4: Jclust = 0.17084139583199587.
Iteration 5: Jclust = 0.17078298358140911.
Iteration 6: Jclust = 0.17078298358140911.
```



In [20]:
```
# Here we convert the vector X of vectors defined above into a matrix XX where the
XX = hcat(X...)'

## Now, we run the new algorithm with the data in matrix format
k = 3;
Cid, Cc, J, cs = newkmeans(XX, k; tol = 5e-3);

# Scatter plot of the clustering solution
p = scatter(title = "newkmeans with 2D data",
    label = " ", legend = :bottomright, size = (350,350), xlims = (-1.5,2.5), ylims
    for i=1:k
        snr = findall(vec(Cid.==i)); # the sample numbers of the i-th cluster
        scatter!(p, XX[snr,1], XX[snr,2], label = string("Cluster ",i))
    end
# Add cluster centers to the scatter plot:
scatter!(p, Cc[:,1],Cc[:,2], marker = :star, markersize = 8, color = :orange, label
display(p)
```




```
Iteration 1: Jclust = 0.24194425269474876.
Iteration 2: Jclust = 0.1708059090034411.
Iteration 3: Jclust = 0.1707829835814091.
```

Ex. D1: Compare the two alogorithms, and comment on their main differences (regarding initialization and convergence/termination).

The two k-means versions do the same basic steps but are different in how they handle the data and how they start and when they stop. The first one (kmeans_) gives random cluster labels at the beginning and uses a strict rule to stop, while the second one (newkmeans) picks random data points as starting centers, stops when the change is small enough, and runs faster because it works with matrices instead of loops.

Ex. D2: Modify the "newkmeans"-algorithm into a version **"anewkmeans"** that compare angles between vectors instead of distances by utilizing the Cosine distances between a row vector (v), and all the rows of a matrix (X) as implemented by the function:

In [21]:
```
allCosDist_(X,v) = 1 .- ((X*v)./(norm(v)*sqrt.(sum(X.^2,dims = 2)))).^2

function anewkmeans(X, k; tol = 5e-3, maxiters = 100)
    # X: data matrix (rows = samples)
    # k: number of clusters
    # tol: tolerance for convergence
    # maxiters: maximum number of iterations

    N, D = size(X)
    D2 = zeros(N, k)          # distance matrix
    C  = zeros(Int, N)        # cluster labels
    cs = zeros(Int, k)        # cluster sizes
    JS = Float64[]            # objective values

    # Randomly pick k starting centers
```

```
    Z = copy(X[randperm(N)[1:k], :])

    Jprev = 1e9
    for iter = 1:maxiters
        # Compute cosine distances from all points to each center
        for i = 1:k
            D2[:, i] = allCosDist_(X, Z[i, :])
        end

        # Assign each sample to the nearest center
        minD2, idx = findmin(D2, dims=2)
        C = vec(getindex.(idx, 2))

        # Compute new objective value
        J = sum(minD2) / N
        push!(JS, J)

        # Check for convergence
        if abs(J - Jprev) / max(Jprev, 1e-12) < tol
            println("Converged at iteration ", iter)
            break
        end
        Jprev = J

        # Update cluster centers (mean of assigned points)
        for i = 1:k
            rows = findall(C .== i)
            cs[i] = length(rows)
            if cs[i] > 0
                Z[i, :] = vec(sum(X[rows, :], dims=1)) / cs[i]
                Z[i, :] ./= norm(Z[i, :])  # normalize center
            end
        end
    end

    return C, Z, JS, cs
end
```

```
anewkmeans (generic function with 1 method)
```

Ex. D3: Apply the **"anewkmeans"-algorithm** to the wikipedia data as described in section 4.4.2 of the VMLS Julia companion (display most frequent words in each cluster and the titles (documents) most similar to the cluster centers):

In [22]:
```
articles, dictionary, titles = wikipedia_data();
art = hcat(articles...)';
N = size(art,1)

k = 5   # number of clusters
Cid, Z, JS, cs = anewkmeans(art, k; tol = 5e-3)

# Display results for each cluster
for j = 1:k
    println("\nCluster: $j (size = $(cs[j]))")
    docs_j = findall(Cid .== j)
```

```
    # Mean word frequencies for this cluster
    mean_words = vec(sum(art[docs_j, :], dims=1)) / length(docs_j)

    # 10 most frequent words
    top_idx = sortperm(mean_words, rev = true)[1:10]
    println("Top words: ", join(dictionary[top_idx], ", "))

    # 3 documents most similar to cluster center
    row_norms = sqrt.(sum(art.^2, dims=2))
    cos_sim = (art * Z[j, :]) ./ (row_norms * norm(Z[j, :]))
    best_docs = sortperm(vec(cos_sim), rev=true)[1:3]
    println("Top articles:")

    for d in best_docs
        println("  ", titles[d])
    end
end
```

```
Converged at iteration 5

Cluster: 1 (size = 114)
Top words: film, star, role, million, release, play, series, appear, character, awar
d
Top articles:
  Leonardo_DiCaprio
  Kate_Beckinsale
  Star_Wars:_The_Force_Awakens

Cluster: 2 (size = 57)
Top words: album, song, release, music, single, band, record, perform, tour, live
Top articles:
  David_Bowie
  Celine_Dion
  Adele

Cluster: 3 (size = 181)
Top words: united, family, president, national, government, american, party, school,
holiday, city
Top articles:
  United_States
  Mahatma_Gandhi
  Marco_Rubio

Cluster: 4 (size = 61)
Top words: match, win, fight, team, event, championship, title, champion, final, def
eat
Top articles:
  Wrestlemania_32
  Night_of_Champions_(2015)
  Payback_(2016)

Cluster: 5 (size = 87)
Top words: season, series, game, character, episode, team, play, player, win, lead
Top articles:
  Game_of_Thrones
  The_X-Files
  American_Horror_Story
```

Ex. D4: How stable are the cluster solutions you obtain for the wikipedia data using a fixed number ($k = 10$) of clusters in the **"anewkmeans"**-algorithm. Discuss/implement an improvenent of your algorithm and compare for the same (wikipedia) data.

The results were sometimes not the same when running the anewkmeans a few times on the Wikipedia data with k = 10.
Some documents changed clusters each time.
This happens because the code starts with random centers,
so the result depends on where it begins. An improvement is to replace random initialization with well spreaded starting centers.
This increases stability and often reduce the number of iterations needed to converge.